



BLAZOR CHEAT SHEET

Core setup, initialization, operations & integration quick reference

PART 1: SETUP & CORE

01. SETUP

Installation

```
dotnet new blazorserver -n MyApp
dotnet new blazorwasm -n MyApp (standalone)
dotnet run
Blazor Server: real-time SignalR connection
Blazor WASM: runs .NET in browser via WebAsse...
dotnet add package Microsoft.AspNetCore.Compo...
```

04. ROUTING

Workflow

```
@page "/counter"
@page "/counter/{StartCount:int}"
[Parameter] public int StartCount { get; set; ...
<NavLink href="/counter" Match=NavLinkMatch.A...
@Inject NavigationManager Nav
Nav.NavigateTo("/home")
Nav.Uri // current URL
```

02. COMPONENTS

Architecture

```
@* Counter.razor *@
<h1>Count: @count</h1>
<button @onclick="Increment">+</button>
@code {
private int count = 0;
private void Increment() => count++;
}
```

05. PARAMETERS & CASCADING

CRUD API

```
@* Child.razor *@
[Parameter] public string Title { get; set; }
[Parameter] public EventCallback<int> OnChang...
[CascadingParameter] public MyTheme Theme { g...
@* Parent usage *@
<Child Title="Hello" OnChange="HandleChange" ...
<CascadingValue Value="theme"><Child /></Casc...
```

03. DATA BINDING

YAML / Config

```
<input @bind="name" /> // two-way bin...
<input @bind-value="name"
@bind-value:event="oninput" />
<p>@name.ToUpper()</p> // one-way fro...
@bind:format="yyyy-MM-dd"
<select @bind="selectedValue">
@foreach (var o in options) { <option>@o</opt...
</select>
```

06. EVENT HANDLING

Integration

```
<button @onclick="HandleClick">Click</button>
<button @onclick="() => count++>Add</button>
<button @onclick="@e => HandleClick(e)">Cli...
private async Task HandleClick(MouseEventArgs...
await JS.InvokeVoidAsync("alert", "clicked!");
}
@onfocus @oninput @onkeydown @onsubmit
```



SECURITY, MONITORING & OPERATIONS

Production hardening, observability, performance tuning & CLI reference

PART 2: OPS & SECURITY

07. LIFECYCLE METHODS

Hardening

```
protected override void OnInitialized()
protected override async Task OnInitializedAs...
protected override void OnParametersSet()
protected override async Task OnParametersSet...
protected override void OnAfterRender(bool fi...
protected override bool ShouldRender() => can...
public void Dispose() { ... } // IDisposable
```

10. FORMS & VALIDATION

Unit Tests

```
<EditForm Model="user" OnValidSubmit="HandleS...
<DataAnnotationsValidator />
<ValidationSummary />
<InputText @bind-Value="user.Name" />
<ValidationMessage For="()" => user.Name" />
<button type="submit">Save</button>
</EditForm>
[Required][MaxLength(50)] public string Name ...
```

08. DEPENDENCY INJECTION

Observability

```
@inject IHttpClientFactory HttpFactory
@inject NavigationManager Nav
@inject IJSRuntime JS
// In Program.cs:
builder.Services.AddScoped<WeatherService>();
builder.Services.AddHttpClient<ApiClient>()
c => c.BaseAddress = new Uri("https://api.mya...
```

11. COMMON GOTCHAS

Commands

Server-side Blazor: network latency on every interaction
Blazor WASM: initial load slow (download .NET runtime)
StateHasChanged() needed for external state changes
Don't use @onclick on non-Blazor elements without @onclick:stopPropagation
IDisposable on components that subscribe to events

09. JS INTEROP

Optimization

```
@inject IJSRuntime JS
await JS.InvokeVoidAsync("console.log", "Hell...
var width = await JS.InvokeAsync<int>("getWin...
// JS calling .NET:
[JSInvokable]
public static string GetData() => "from .NET";
// In JS: DotNet.invokeMethodAsync('MyApp', 'G...
```